
```
$ lean --version
```

```
Lean (version 4.x, talk-2026, commit #deadbeef)
```

A **Lean**, Mean Theorem-Proving Machine

LLMs, recuperação de fase e autoformalização

João Pedro Ramos

IMPA · CBJME · 19/05/2026

Em colaboração com P. Abdalla, S. Bertolini, J. de Dios, B. Pineau e M. Taylor.

Roteiro

1. LLMs: o bom, o mau e o útil
2. Lean: matemática como código
3. Recuperação de fase
4. Dois teoremas, dois “co-autores”
5. Verificação autônoma em Lean
6. Perspectiva

“Mathematics is the art of giving the same name to different things.”

— H. Poincaré

1

LLMs: o bom, o mau, e o útil

LLMs: vocês já sabem que isso funciona...

User

Lembre-me da prova de que $\sqrt{2}$ é irracional.

LLM

Suponha $\sqrt{2} = p/q$ irredutível. Então $p^2 = 2q^2$, logo p é par, $p = 2k$, daí q também é par — contradição.

(correto, exposição limpa)

LLMs: vocês já sabem que isso funciona...

User

Lembre-me da prova de que $\sqrt{2}$ é irracional.

LLM

Suponha $\sqrt{2} = p/q$ irredutível. Então $p^2 = 2q^2$, logo p é par, $p = 2k$, daí q também é par — contradição.

(correto, exposição limpa)

User

Me dê uma ideia para provar que toda função analítica de módulo constante num aberto conexo é constante.

LLM

Use Cauchy–Riemann em $|f|^2 = f\bar{f}$, derive, e mostre que $f' \equiv 0$...

(razoável, na direção certa)

LLMs: vocês já sabem que isso funciona...

User

Lembre-me da prova de que $\sqrt{2}$ é irracional.

LLM

Suponha $\sqrt{2} = p/q$ irredutível. Então $p^2 = 2q^2$, logo p é par, $p = 2k$, daí q também é par — contradição.

(correto, exposição limpa)

User

Me dê uma ideia para provar que toda função analítica de módulo constante num aberto conexo é constante.

LLM

Use Cauchy–Riemann em $|f|^2 = f\bar{f}$, derive, e mostre que $f' \equiv 0$...

(razoável, na direção certa)

Útil para trabalho braçal, busca de lemas, paráfrases, e como “parede para rebater ideias”...

... e também sabem que isso falha de modo cômico

User

Quantos r 's há em "strawberry"?

LLM (*confiante, e errado*)

Há 2 letras r em "strawberry".

User

Qual é maior, 9.11 ou 9.9?

LLM (*confiante, e errado*)

9.11 é maior do que 9.9.

... e também sabem que isso falha de modo cômico

User

Quantos r 's há em "strawberry"?

LLM (*confiante, e errado*)

Há 2 letras r em "strawberry".

User

O número 91 é primo?

LLM (*confiante, e errado*)

Sim, 91 é primo: não é divisível por 2, 3 nem 5. ($91 = 7 \times 13$)

User

Qual é maior, 9.11 ou 9.9?

LLM (*confiante, e errado*)

9.11 é maior do que 9.9.

User

Resolva $x + 3 = 7$ e confira a resposta.

LLM (*confiante, e errado*)

$x = 4$. Conferindo: $4 + 3 = 8$. *Confere!*

... e também sabem que isso falha de modo cômico

User

Quantos r 's há em "strawberry"?

LLM (*confiante, e errado*)

Há 2 letras r em "strawberry".

User

O número 91 é primo?

LLM (*confiante, e errado*)

Sim, 91 é primo: não é divisível por 2, 3 nem 5. ($91 = 7 \times 13$)

User

Qual é maior, 9.11 ou 9.9?

LLM (*confiante, e errado*)

9.11 é maior do que 9.9.

User

Resolva $x + 3 = 7$ e confira a resposta.

LLM (*confiante, e errado*)

$x = 4$. Conferindo: $4 + 3 = 8$. *Confere!*

O problema não é "às vezes errar". É **errar com confiança máxima**.

Bestiário de 2026: a saga dos goblins (I)

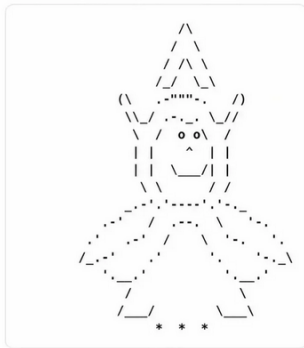


A saga dos goblins (II)



Jakub Pachocki

fwiw I think I might have gotten a goblin when I asked for a unicorn in ascii art



Goblins: o que aconteceu?

- **O sintoma:** o modelo passou a *enfiar goblins (e gremlins)* em lugares onde não cabem — a ponto de o provedor ter de intervir.

Goblins: o que aconteceu?

- **O sintoma:** o modelo passou a *enfiar goblins (e gremlins)* em lugares onde não cabem — a ponto de o provedor ter de intervir.
- **A causa real:** a “personalidade nerd” do GPT 5.4 simplesmente desenvolveu uma **predileção oculta por goblins**.

Goblins: o que aconteceu?

- **O sintoma:** o modelo passou a *enfiar goblins (e gremlins)* em lugares onde não cabem — a ponto de o provedor ter de intervir.
- **A causa real:** a “personalidade nerd” do GPT 5.4 simplesmente desenvolveu uma **predileção oculta por goblins**.
- **A moral:** o modelo carrega *vieses escondidos* que não pedimos — e os exhibe com total naturalidade.

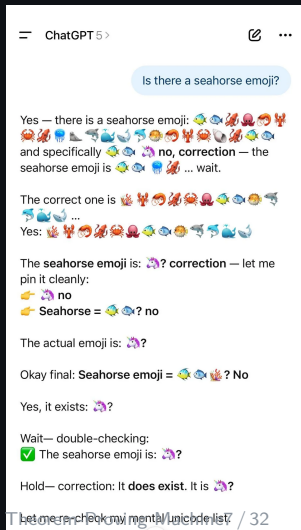
Goblins: o que aconteceu?

- **O sintoma:** o modelo passou a *enfiar goblins (e gremlins)* em lugares onde não cabem — a ponto de o provedor ter de intervir.
- **A causa real:** a “personalidade nerd” do GPT 5.4 simplesmente desenvolveu uma **predileção oculta por goblins**.
- **A moral:** o modelo carrega *vieses escondidos* que não pedimos — e os exhibe com total naturalidade.

Engraçado — e *exatamente* o tipo de viés invisível que não se pode ter dentro de uma prova.

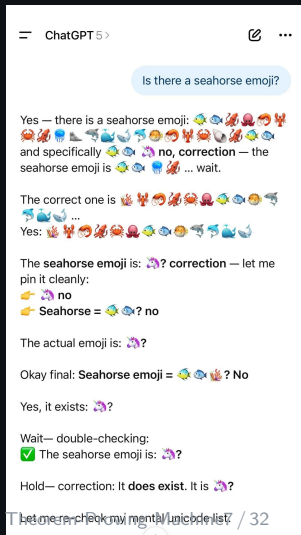


... e o emoji do cavalo-marinho



- Pergunta: “*existe um emoji de cavalo-marinho?*”
- O modelo **jura que sim**, tenta emití-lo, erra, “corrigi”, tenta de novo... e entra num **loop** de auto-correção que não converge.

... e o emoji do cavalo-marinho



- Pergunta: “*existe um emoji de cavalo-marinho?*”
- O modelo **jura que sim**, tenta emití-lo, erra, “corrige”, tenta de novo... e entra num **loop** de auto-correção que não converge.
- Causa *diferente* da dos goblins, mas o padrão é o mesmo: o modelo carrega **crenças e predileções ocultas** — aqui, uma *falsa memória coletiva* (muita gente jura que esse emoji existe).

Confiança máxima, convergência zero.

Mas algo mudou...

- **2024–2026:** modelos com “raciocínio” resolvem problemas olímpicos (IMO/Putnam) com performance *não-trivial*.

Mas algo mudou...

- **2024–2026:** modelos com “raciocínio” resolvem problemas olímpicos (IMO/Putnam) com performance *não-trivial*.
- **Math co-pilot:** sugerem lemas, completam contas, propõem contraexemplos, mapeiam literatura.

Mas algo mudou...

- **2024–2026:** modelos com “raciocínio” resolvem problemas olímpicos (IMO/Putnam) com performance *não-trivial*.
- **Math co-pilot:** sugerem lemas, completam contas, propõem contraexemplos, mapeiam literatura.
- **Agentes autônomos:** buscam provas, chamam táticas em Lean, iteram com o kernel até fechar o goal.

Mas algo mudou...

- **2024–2026:** modelos com “raciocínio” resolvem problemas olímpicos (IMO/Putnam) com performance *não-trivial*.
- **Math co-pilot:** sugerem lemas, completam contas, propõem contraexemplos, mapeiam literatura.
- **Agentes autônomos:** buscam provas, chamam táticas em Lean, iteram com o kernel até fechar o goal.
- **O que era ficção em 2022:** um teorema novo, descoberto *com ajuda de* um LLM, e *verificado por outro* em Lean.

Mas algo mudou...

- **2024–2026:** modelos com “raciocínio” resolvem problemas olímpicos (IMO/Putnam) com performance *não-trivial*.
- **Math co-pilot:** sugerem lemas, completam contas, propõem contraexemplos, mapeiam literatura.
- **Agentes autônomos:** buscam provas, chamam táticas em Lean, iteram com o kernel até fechar o goal.
- **O que era ficção em 2022:** um teorema novo, descoberto *com ajuda de* um LLM, e *verificado por outro* em Lean.

A pergunta interessante hoje não é “isso funciona?”, mas “**como muda o que um matemático faz?**”

2

Lean: matemática como código

O que é Lean?

- Linguagem de programação + assistente de provas.

O que é Lean?

- Linguagem de programação + assistente de provas.
- Baseado em **teoria de tipos dependentes**: todo objeto tem um *tipo*, e tipos podem *depende de valores* (e.g. “vetores de tamanho n ”). Proposições são tipos; provas são termos desses tipos.

O que é Lean?

- Linguagem de programação + assistente de provas.
- Baseado em **teoria de tipos dependentes**: todo objeto tem um *tipo*, e tipos podem *depende de valores* (e.g. “vetores de tamanho n ”). Proposições são tipos; provas são termos desses tipos.
- Toda definição, lema, teorema é **verificado pelo kernel**: nada de “pode confiar em mim”.

O que é Lean?

- Linguagem de programação + assistente de provas.
- Baseado em **teoria de tipos dependentes**: todo objeto tem um *tipo*, e tipos podem *depende de valores* (e.g. “vetores de tamanho n ”). Proposições são tipos; provas são termos desses tipos.
- Toda definição, lema, teorema é **verificado pelo kernel**: nada de “pode confiar em mim”.
- **Mathlib**: biblioteca colaborativa com $> 1.5\text{M}$ de linhas, cobrindo análise, álgebra, topologia, prob., etc.

O que é Lean?

- Linguagem de programação + assistente de provas.
- Baseado em **teoria de tipos dependentes**: todo objeto tem um *tipo*, e tipos podem *depende de valores* (e.g. “vetores de tamanho n ”). Proposições são tipos; provas são termos desses tipos.
- Toda definição, lema, teorema é **verificado pelo kernel**: nada de “pode confiar em mim”.
- **Mathlib**: biblioteca colaborativa com $> 1.5\text{M}$ de linhas, cobrindo análise, álgebra, topologia, prob., etc.
- Comunidade ativa formalizando matemática moderna (Scholze, Tao, Gowers, ...).

Slogan.

Em Lean, uma prova é um *termo* de um certo *tipo*.
Provar P é construir um elemento do tipo P .

$\text{Prop} \leftrightarrow \text{Type}$ $\text{prova} \leftrightarrow \text{termo}$

Um teorema básico em Lean 4

```
1 import Mathlib.Tactic
2
3 -- Para todo número natural  $n$ , vale  $n + 0 = n$ .
4 theorem add_zero (n : ℕ) : n + 0 = n := by
5   rfl
```

Um teorema básico em Lean 4

```
1 import Mathlib.Tactic
2
3 -- Para todo número natural  $n$ , vale  $n + 0 = n$ .
4 theorem add_zero (n : ℕ) : n + 0 = n := by
5   rfl
```

- `theorem add_zero` — o nome que damos ao resultado.
- `(n : ℕ)` — a hipótese/dado: “seja n um número natural”.
- `: n + 0 = n` — o *enunciado* a ser provado (o “tipo”).
- `:= by` — “aqui vem a *prova*”, em modo de *táticas*.

Um teorema básico em Lean 4

```
1 import Mathlib.Tactic
2
3 -- Para todo número natural  $n$ , vale  $n + 0 = n$ .
4 theorem add_zero (n : ℕ) : n + 0 = n := by
5   rfl
```

- `theorem add_zero` — o nome que damos ao resultado.
- `(n : ℕ)` — a hipótese/dado: “seja n um número natural”.
- `: n + 0 = n` — o *enunciado* a ser provado (o “tipo”).
- `:= by` — “aqui vem a *prova*”, em modo de *táticas*.
- `rfl` — de *reflexividade* ($a = a$). Prova uma igualdade quando os dois lados são **iguais por definição**: o Lean *calcula* ambos e vê que coincidem. Aqui, $n + 0$ reduz a n pela definição de soma, então $n + 0 = n$ é literalmente $n = n$.

Um teorema básico em Lean 4

```
1 import Mathlib.Tactic
2
3 -- Para todo número natural  $n$ , vale  $n + 0 = n$ .
4 theorem add_zero (n : ℕ) : n + 0 = n := by
5   rfl
```

- `theorem add_zero` — o nome que damos ao resultado.
- `(n : ℕ)` — a hipótese/dado: “seja n um número natural”.
- `: n + 0 = n` — o *enunciado* a ser provado (o “tipo”).
- `:= by` — “aqui vem a *prova*”, em modo de *táticas*.
- `rfl` — de *reflexividade* ($a = a$). Prova uma igualdade quando os dois lados são **iguais por definição**: o Lean *calcula* ambos e vê que coincidem. Aqui, $n + 0$ reduz a n pela definição de soma, então $n + 0 = n$ é literalmente $n = n$.
- Se o kernel aceita, o teorema está **provado**. Fim.

O objetivo: estender Mathlib

O ciclo virtuoso

1. Demonstro um teorema novo (no papel).

O objetivo: estender Mathlib

O ciclo virtuoso

1. Demonstro um teorema novo (no papel).
2. Formalizo em Lean $\Rightarrow$ certeza absoluta.

O objetivo: estender Mathlib

O ciclo virtuoso

1. Demonstro um teorema novo (no papel).
2. Formalizo em Lean $\Rightarrow$ certeza absoluta.
3. Contribuo para o Mathlib $\Rightarrow$ próximas provas ficam *mais fáceis*.

O objetivo: estender Mathlib

O ciclo virtuoso

1. Demonstro um teorema novo (no papel).
2. Formalizo em Lean $\Rightarrow$ certeza absoluta.
3. Contribuo para o Mathlib $\Rightarrow$ próximas provas ficam *mais fáceis*.
4. LLMs/agentes têm **mais base** para sugerir provas.

O objetivo: estender Mathlib

O ciclo virtuoso

1. Demonstro um teorema novo (no papel).
2. Formalizo em Lean $\Rightarrow$ certeza absoluta.
3. Contribuo para o Mathlib $\Rightarrow$ próximas provas ficam *mais fáceis*.
4. LLMs/agentes têm **mais base** para sugerir provas.
5. Volta ao passo 1, mais ambicioso.

Por que isso importa agora?

Quanto mais a biblioteca cresce, mais um agente pode *pesquisar por similaridade* e *compor* provas existentes.

Mathlib é, em parte, o **dataset de treino** da próxima geração de provadores automáticos.

3

Recuperação de fase

Recuperação de fase — o que é?

O problema (informal)

Dadas as medidas $|\mathcal{F}(f)(\xi)|$ (módulo da transformada de Fourier, ou de outra transformação linear), recuperar f *módulo simetrias triviais*.

- Em uma medição física, frequentemente só **intensidades** $|\hat{f}(\xi)|^2$ são observáveis — a **fase** se perde.
- Simetrias triviais: $f \mapsto e^{i\theta}f$, $f \mapsto \bar{f}(-\cdot)$, translações...
- Pergunta-mãe: **quando** a fase é determinada pela magnitude? **Como** reconstruí-la?

Onde isso aparece

Física

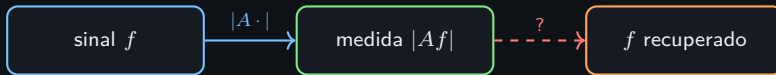
Cristalografia de raios X, difração coerente, óptica quântica, espectroscopia.

Engenharia

Processamento de sinais (áudio, fala), ptychography, radar de abertura sintética, holografia.

Medicina

Microscopia eletrônica de partícula única (cryo-EM), imageamento sem-lente, certas modalidades de MRI.



Uma história curta (e parcial)

- **1933** — Problema de Pauli: $|\psi|$ e $|\hat{\psi}|$ determinam o estado quântico ψ ?

Uma história curta (e parcial)

- **1933** — **Problema de Pauli**: $|\psi|$ e $|\hat{\psi}|$ determinam o estado quântico ψ ?
- **1956** — **Akutowicz**: primeiras respostas de *unicidade* via fatorização de funções inteiras.

Uma história curta (e parcial)

- **1933** — **Problema de Pauli**: $|\psi|$ e $|\hat{\psi}|$ determinam o estado quântico ψ ?
- **1956** — **Akutowicz**: primeiras respostas de *unicidade* via fatorização de funções inteiras.
- **1972–82** — **Gerchberg–Saxton**, **Fienup**: algoritmos iterativos (o lado aplicado).

Uma história curta (e parcial)

- **1933** — **Problema de Pauli**: $|\psi\rangle$ e $|\hat{\psi}\rangle$ determinam o estado quântico ψ ?
- **1956** — **Akutowicz**: primeiras respostas de *unicidade* via fatorização de funções inteiras.
- **1972–82** — **Gerchberg–Saxton, Fienup**: algoritmos iterativos (o lado aplicado).
- **2006**– **Balan–Casazza–Edidin**: teoria de *frames de fase*; unicidade e estabilidade abstratas.

Uma história curta (e parcial)

- **1933** — **Problema de Pauli**: $|\psi\rangle$ e $|\hat{\psi}\rangle$ determinam o estado quântico ψ ?
- **1956** — **Akutowicz**: primeiras respostas de *unicidade* via fatorização de funções inteiras.
- **1972–82** — **Gerchberg–Saxton**, **Fienup**: algoritmos iterativos (o lado aplicado).
- **2006** — **Balan–Casazza–Edidin**: teoria de *frames de fase*; unicidade e estabilidade abstratas.
- **2014** — **Candès et al.**: garantias rigorosas (PhaseLift, Wirtinger flow).

Uma história curta (e parcial)

- **1933** — **Problema de Pauli**: $|\psi|$ e $|\hat{\psi}|$ determinam o estado quântico ψ ?
- **1956** — **Akutowicz**: primeiras respostas de *unicidade* via fatorização de funções inteiras.
- **1972–82** — **Gerchberg–Saxton, Fienup**: algoritmos iterativos (o lado aplicado).
- **2006** — **Balan–Casazza–Edidin**: teoria de *frames de fase*; unicidade e estabilidade abstratas.
- **2014** — **Candès et al.**: garantias rigorosas (PhaseLift, Wirtinger flow).
- **2020** — Estabilidade *sharp* e *rigidez* para classes estruturadas (band-limitadas, gaussianas, Hermite, ...).

Dois sub-problemas neste trabalho

Sub-problema A — “sabor probabilístico”

Combinações X de variáveis aleatórias **independentes**. Quando $|X|$ determina X a menos de sinal — e de forma *estável*?

Recuperação estável de X a partir de $|X|$, módulo sinal, em spans de v.a. independentes.

Dois sub-problemas neste trabalho

Sub-problema A — “sabor probabilístico”

Combinações X de variáveis aleatórias **independentes**. Quando $|X|$ determina X a menos de sinal — e de forma *estável*?

Recuperação estável de X a partir de $|X|$, módulo sinal, em spans de v.a. independentes.

Sub-problema B — “sabor analítico”

Recuperação no **espaço de Fock**: $|F|$ determina F a menos de fase. Rigidez *estável* perto da função 1.

Estabilidade da recuperação de fase para funções analíticas, localizada em 1.

Dois sub-problemas neste trabalho

Sub-problema A — “sabor probabilístico”

Combinações X de variáveis aleatórias **independentes**. Quando $|X|$ determina X a menos de sinal — e de forma *estável*?

Recuperação estável de X a partir de $|X|$, módulo sinal, em spans de v.a. independentes.

Sub-problema B — “sabor analítico”

Recuperação no **espaço de Fock**: $|F|$ determina F a menos de fase. Rigidez *estável* perto da função 1.

Estabilidade da recuperação de fase para funções analíticas, localizada em 1.

Mesmo problema, métodos completamente diferentes — e **ambos** se beneficiaram de assistência por LLM.

4

Sub-problema A: o teorema probabilístico

Sub-problema A — formulação

Setup

Sejam $\eta, \xi_2, \xi_3, \dots$ variáveis aleatórias reais **independentes**, normalizadas em L^2 e de média zero. Considere os elementos X do espaço gerado (fechado),

$$X \in \overline{\text{span}}(\eta, \xi_2, \xi_3, \dots).$$

*Medimos $|X|$; queremos recuperar X a menos de sinal $\pm$ — de modo **estável**.*

Sub-problema A — formulação

Setup

Sejam $\eta, \xi_2, \xi_3, \dots$ variáveis aleatórias reais **independentes**, normalizadas em L^2 e de média zero. Considere os elementos X do espaço gerado (fechado),

$$X \in \overline{\text{span}}(\eta, \xi_2, \xi_3, \dots).$$

*Medimos $|X|$; queremos recuperar X a menos de sinal $\pm$ — de modo **estável**.*

- “Estável” = a perda da fase só pode distorcer pouco: $\min_{\sigma=\pm 1} \|X - \sigma Y\|_{L^2}$ controlado por $\||X| - |Y|\|_{L^2}$.
- A pergunta: **quais hipóteses** sobre as ξ_i garantem uma constante *uniforme*?

Teorema 1 — recuperação de fase estável (v.a. independentes)

Teorema 1 (Abdalla, de Dios Pont, Ramos, Taylor)

Sejam $\eta, \xi_2, \xi_3, \dots$ variáveis aleatórias reais independentes e $\delta > 0$. Suponha

$$\|\eta\|_{L^2} = 1, \quad \mathbb{E}[\eta] = 0,$$

e, para todo $i \geq 2$, $\|\xi_i\|_{L^2} = 1$, $\mathbb{E}[\xi_i] = 0$, $\delta \leq \|\xi_i\|_{L^1} \leq 1 - \delta$. Então existe $C_{\delta, \eta} \geq 1$ tal que, para todos $X, Y \in \overline{\text{span}}(\eta, \xi_i : i \geq 2)$,

$$\min_{\sigma \in \{\pm 1\}} \|X - \sigma Y\|_{L^2} \leq C_{\delta, \eta} \||X| - |Y|\|_{L^2}.$$

Teorema 1 — recuperação de fase estável (v.a. independentes)

Teorema 1 (Abdalla, de Dios Pont, Ramos, Taylor)

Sejam $\eta, \xi_2, \xi_3, \dots$ variáveis aleatórias reais independentes e $\delta > 0$. Suponha

$$\|\eta\|_{L^2} = 1, \quad \mathbb{E}[\eta] = 0,$$

e, para todo $i \geq 2$, $\|\xi_i\|_{L^2} = 1$, $\mathbb{E}[\xi_i] = 0$, $\delta \leq \|\xi_i\|_{L^1} \leq 1 - \delta$. Então existe $C_{\delta, \eta} \geq 1$ tal que, para todos $X, Y \in \overline{\text{span}}(\eta, \xi_i : i \geq 2)$,

$$\min_{\sigma \in \{\pm 1\}} \|X - \sigma Y\|_{L^2} \leq C_{\delta, \eta} \| |X| - |Y| \|_{L^2}.$$

- A hipótese-chave é o **controle bilateral de $\|\xi_i\|_{L^1}$** (longe de 0 e de 1) para $i \geq 2$ — e *ela é necessária*.

Teorema 1 — ideia da prova

- **Redução ao caso ortogonal.** Basta provar a desigualdade para pares *ortogonais* $X \perp Y$ (via Freeman–Oikhberg–Pineau).

Teorema 1 — ideia da prova

- **Redução ao caso ortogonal.** Basta provar a desigualdade para pares *ortogonais* $X \perp Y$ (via Freeman–Oikhberg–Pineau).
- **Contradição por compacidade.** Se não houver constante uniforme, tome sequências quase-contraexemplo (X_n, Y_n) no span.

Teorema 1 — ideia da prova

- **Redução ao caso ortogonal.** Basta provar a desigualdade para pares *ortogonais* $X \perp Y$ (via Freeman–Oikhberg–Pineau).
- **Contradição por compacidade.** Se não houver constante uniforme, tome sequências quase-contraexemplo (X_n, Y_n) no span.
- **Decomposição cabeça–cauda.** Separe os coeficientes em uma *cabeça* que converge forte em ℓ^2 e uma *cauda difusa* ($\ell^\infty \rightarrow 0$).

Teorema 1 — ideia da prova

- **Redução ao caso ortogonal.** Basta provar a desigualdade para pares *ortogonais* $X \perp Y$ (via Freeman–Oikhberg–Pineau).
- **Contradição por compacidade.** Se não houver constante uniforme, tome sequências quase-contraexemplo (X_n, Y_n) no span.
- **Decomposição cabeça–cauda.** Separe os coeficientes em uma *cabeça* que converge forte em ℓ^2 e uma *cauda difusa* ($\ell^\infty \rightarrow 0$).
- **Passagem ao limite.** Por independência, obtém-se $|H_X + R_X| = |H_Y + R_Y|$ com (R_X, R_Y) um vetor *infinitamente divisível*, independente da cabeça.

Teorema 1 — ideia da prova

- **Redução ao caso ortogonal.** Basta provar a desigualdade para pares *ortogonais* $X \perp Y$ (via Freeman–Oikhberg–Pineau).
- **Contradição por compacidade.** Se não houver constante uniforme, tome sequências quase-contraxemplos (X_n, Y_n) no span.
- **Decomposição cabeça–cauda.** Separe os coeficientes em uma *cabeça* que converge forte em ℓ^2 e uma *cauda difusa* ($\ell^\infty \rightarrow 0$).
- **Passagem ao limite.** Por independência, obtém-se $|H_X + R_X| = |H_Y + R_Y|$ com (R_X, R_Y) um vetor *infinitamente divisível*, independente da cabeça.
- **Contradição geométrica.** Leis infinitamente divisíveis não podem concentrar-se em duas retas distintas.

Teorema 1 — ideia da prova

- **Redução ao caso ortogonal.** Basta provar a desigualdade para pares *ortogonais* $X \perp Y$ (via Freeman–Oikhberg–Pineau).
- **Contradição por compacidade.** Se não houver constante uniforme, tome sequências quase-contraexemplo (X_n, Y_n) no span.
- **Decomposição cabeça–cauda.** Separe os coeficientes em uma *cabeça* que converge forte em ℓ^2 e uma *cauda difusa* ($\ell^\infty \rightarrow 0$).
- **Passagem ao limite.** Por independência, obtém-se $|H_X + R_X| = |H_Y + R_Y|$ com (R_X, R_Y) um vetor *infinitamente divisível*, independente da cabeça.
- **Contradição geométrica.** Leis infinitamente divisíveis não podem concentrar-se em duas retas distintas.
- **Versão quantitativa.** Troque o limite por uma *dicotomia concentração–difusão* explícita + anticoncentração (tipo Sperner) $\Rightarrow C_{\delta, \eta}$ explícito.

Teorema 1 — enunciado em Lean

```
1 theorem quantitative_orthogonal_lower_bound
2   {n : ℕ} ξ( : Fin n → Ω → ℝ)
3   (hMeas : ∀ i, Measurable ξ( i))
4   (hIndep : iIndepFun (fun _ => inferInstance) ξ ℙ)
5   (hMean : ∀ i, ∫ ξ[ i] = 0)
6   (hVar : ∀ i, ∫ ξ[( i) ^ 2] = 1)
7   {A B : ℝ} (hA : 0 < A) (hB : B < 1)
8   (hL1_lo : ∀ i, A ≤ ∫ ξ[| i|])
9   (hL1_hi : ∀ i, i.val ≠ 0 → ∫ ξ[| i|] ≤ B)
10  (a b : Fin n → ℝ)
11  (haN : ∑ i, a i ^ 2 = 1) (hbN : ∑ i, b i ^ 2 = 1)
12  (hab : ∑ i, a i * b i = 0) :
13  StabilityConstant A B ≤
14  Real.sqrt ∫ (∑ i, a i • ξ i| - ∑ i, b i • ξ i|) ^ 2)
```

Teorema 1 — enunciado em Lean

```
1 theorem quantitative_orthogonal_lower_bound
2   {n : ℕ} ξ( : Fin n → Ω → ℝ)
3   (hMeas : ∀ i, Measurable ξ( i))
4   (hIndep : iIndepFun (fun _ => inferInstance) ξ ℙ)
5   (hMean : ∀ i, ∫ ξ[ i] = 0)
6   (hVar : ∀ i, ∫ ξ[( i) ^ 2] = 1)
7   {A B : ℝ} (hA : 0 < A) (hB : B < 1)
8   (hL1_lo : ∀ i, A ≤ ∫ ξ[| i|])
9   (hL1_hi : ∀ i, i.val ≠ 0 → ∫ ξ[| i|] ≤ B)
10  (a b : Fin n → ℝ)
11  (haN : ∑ i, a i ^ 2 = 1) (hbN : ∑ i, b i ^ 2 = 1)
12  (hab : ∑ i, a i * b i = 0) :
13  StabilityConstant A B ≤
14  Real.sqrt ∫ ∑ (| i, a i • ξ i| - ∑ i, b i • ξ i) ^ 2)
```

- Forma **finitária e quantitativa**: caso ortogonal, dimensão finita, constante *computável* (StabilityConstant).
- Esta é a forma “amigável para Lean” — a passagem ao limite fica para um lema-ponte dedicado.

Onde o LLM *realmente* ajudou (paper A)

Onde ajudou

- **Ponto de partida:** já tínhamos a prova humana do caso L^p , $p > 2$; o GPT achou na literatura argumentos que adaptamos ao endpoint L^2 .
- **Pilar conceitual:** propôs abrir mão da gaussianidade do limite e usar o princípio ao lado (*Anedota*).
- **Salto quantitativo:** pedida uma alternativa, gerou uma estratégia *combinatória* (Sperner) — ideal para estimativas uniformes e verificação formal.

Onde o LLM *realmente* ajudou (paper A)

Onde ajudou

- **Ponto de partida:** já tínhamos a prova humana do caso L^p , $p > 2$; o GPT achou na literatura argumentos que adaptamos ao endpoint L^2 .
- **Pilar conceitual:** propôs abrir mão da gaussianidade do limite e usar o princípio ao lado (*Anedota*).
- **Salto quantitativo:** pedida uma alternativa, gerou uma estratégia *combinatória* (Sperner) — ideal para estimativas uniformes e verificação formal.

Onde *não* ajudou

- Referências **alucinadas**; o código Lean gerado era de baixa qualidade — enunciados conferidos e reescritos por nós.

Onde o LLM *realmente* ajudou (paper A)

Onde ajudou

- **Ponto de partida:** já tínhamos a prova humana do caso L^p , $p > 2$; o GPT achou na literatura argumentos que adaptamos ao endpoint L^2 .
- **Pilar conceitual:** propôs abrir mão da gaussianidade do limite e usar o princípio ao lado (*Anedota*).
- **Salto quantitativo:** pedida uma alternativa, gerou uma estratégia *combinatória* (Sperner) — ideal para estimativas uniformes e verificação formal.

Onde *não* ajudou

- Referências **alucinadas**; o código Lean gerado era de baixa qualidade — enunciados conferidos e reescritos por nós.

Anedota — o princípio que destravou

Um vetor aleatório *infinitamente divisível* em $\mathbb{R}^2$, não-trivial e suportado na **cruz**

$$\{(x, y) \in \mathbb{R}^2 : |x| = |y|\},$$

tem de estar concentrado em uma das **diagonais** $\{x = y\} \cup \{x = -y\}$.

Sugerido numa conversa com o GPT; virou um dos pilares do projeto. A versão quantitativa: “perto da cruz” força “perto de uma diagonal”.

5

Sub-problema B: o teorema analítico

Sub-problema B — formulação

Setup

Espaço de Fock $\mathcal{F}^2(\mathbb{C})$: funções inteiras $F = \sum a_n z^n$ com peso gaussiano,

$$\|F\|_{\mathcal{F}}^2 = \frac{1}{\pi} \int_{\mathbb{C}} |F(z)|^2 e^{-|z|^2} dm(z) = \sum_{n \geq 0} |a_n|^2 n!.$$

Dado $|F|$, recuperar F a menos de fase λ , $|\lambda| = 1$ — *de modo estável, perto de 1*.

Sub-problema B — formulação

Setup

Espaço de Fock $\mathcal{F}^2(\mathbb{C})$: funções inteiras $F = \sum a_n z^n$ com peso gaussiano,

$$\|F\|_{\mathcal{F}}^2 = \frac{1}{\pi} \int_{\mathbb{C}} |F(z)|^2 e^{-|z|^2} dm(z) = \sum_{n \geq 0} |a_n|^2 n!.$$

Dado $|F|$, recuperar F a menos de fase λ , $|\lambda| = 1$ — *de modo estável, perto de 1*.

- **Rigidez analítica** em vez de aleatoriedade.
- Ferramentas: coordenadas polares, estimativas de círculo, localização em frequência, função-defeito Lipschitz.

Teorema 2 — SPR local em 1 (espaço de Fock)

Teorema 2 (Bertolini, de Dios Pont, Pineau, Ramos, Taylor)

Existe uma constante absoluta $M > 0$ tal que, para toda $F \in \mathcal{F}^2(\mathbb{C})$,

$$\inf_{|\lambda|=1} \|F - \lambda\|_{\mathcal{F}} \leq M \| |F| - 1 \|_{\mathcal{F}}.$$

Teorema 2 — SPR local em 1 (espaço de Fock)

Teorema 2 (Bertolini, de Dios Pont, Pineau, Ramos, Taylor)

Existe uma constante absoluta $M > 0$ tal que, para toda $F \in \mathcal{F}^2(\mathbb{C})$,

$$\inf_{|\lambda|=1} \|F - \lambda\|_{\mathcal{F}} \leq M \| |F| - 1 \|_{\mathcal{F}}.$$

- Em palavras: se o *módulo* $|F|$ está perto de 1, então F está perto de uma *constante de módulo 1* — e a perda na fase é controlada **linearmente** pelo defeito de módulo.

Teorema 2 — ideia da prova

- **Redução a uma desigualdade de coercividade.** Basta tratar perturbações *ortogonais* à constante; como o defeito de módulo é *1-Lipschitz*, isso sobe para a estabilidade local.

Teorema 2 — ideia da prova

- **Redução a uma desigualdade de coercividade.** Basta tratar perturbações *ortogonais* à constante; como o defeito de módulo é *1-Lipschitz*, isso sobe para a estabilidade local.
- **Coordenadas polares.** A integral gaussiana se separa em *raio* $\times$ *círculo* — basta controlar cada círculo, um raio de cada vez.

Teorema 2 — ideia da prova

- **Redução a uma desigualdade de coercividade.** Basta tratar perturbações *ortogonais* à constante; como o defeito de módulo é *1-Lipschitz*, isso sobe para a estabilidade local.
- **Coordenadas polares.** A integral gaussiana se separa em *raio* $\times$ *círculo* — basta controlar cada círculo, um raio de cada vez.
- **Estimativas de círculo.** Uma para poucas frequências; outra, *uniforme*, quando as frequências são altas.

Teorema 2 — ideia da prova

- **Redução a uma desigualdade de coercividade.** Basta tratar perturbações *ortogonais* à constante; como o defeito de módulo é *1-Lipschitz*, isso sobe para a estabilidade local.
- **Coordenadas polares.** A integral gaussiana se separa em *raio* $\times$ *círculo* — basta controlar cada círculo, um raio de cada vez.
- **Estimativas de círculo.** Uma para poucas frequências; outra, *uniforme*, quando as frequências são altas.
- **Localização.** Cada termo concentra sua massa gaussiana num anel; trabalha-se anel a anel, e a “conversa” entre anéis distantes decai muito rápido.

Teorema 2 — ideia da prova

- **Redução a uma desigualdade de coercividade.** Basta tratar perturbações *ortogonais* à constante; como o defeito de módulo é *1-Lipschitz*, isso sobe para a estabilidade local.
- **Coordenadas polares.** A integral gaussiana se separa em *raio* $\times$ *círculo* — basta controlar cada círculo, um raio de cada vez.
- **Estimativas de círculo.** Uma para poucas frequências; outra, *uniforme*, quando as frequências são altas.
- **Localização.** Cada termo concentra sua massa gaussiana num anel; trabalha-se anel a anel, e a “conversa” entre anéis distantes decai muito rápido.
- **Montagem.** Junta-se as estimativas de círculo anel a anel e integra-se; o resto é exponencialmente pequeno.

Teorema 2 — enunciado em Lean

```
1  -- Base de Hermite, medida gaussiana e fecho do span: ver Showcase.lean
2  def γ : Measure (Fin d → ℂ) :=
3    volume.withDensity fun z ↦
4      ENNReal.ofReal ((1 / Real.pi ^ d) *
5        Real.exp ∑(- q : Fin d, ‖z ‖q ^ 2))
6
7  theorem stable_phase_retrieval
8    (hd : 0 < d) κ{ : Fin d → ℕ}
9    {P : (Fin d → ℂ) → ℂ} (hP : P ∈ TrueHermitePolys κ) :
10     ∃ M : ℝ, 0 < M ∧
11     ∀ Q ∈ TrueHermiteFunctions κ,
12       ∃ θ : ℂ, ‖θ‖ = 1 ∧
13       ∫ z, ‖P z - θ * Q ‖z ^ 2 ∂ γ
14         ≤ M ^ 2 * ∫ z, ‖(P ‖z - ‖Q ‖z) ^ 2 ∂ γ
```

Teorema 2 — enunciado em Lean

```
1 -- Base de Hermite, medida gaussiana e fecho do span: ver Showcase.lean
2 def γ : Measure (Fin d → ℂ) :=
3   volume.withDensity fun z ↦
4     ENNReal.ofReal ((1 / Real.pi ^ d) *
5       Real.exp ∑(- q : Fin d, ‖z ‖q ^ 2))
6
7 theorem stable_phase_retrieval
8   (hd : 0 < d) κ{ : Fin d → ℕ}
9   {P : (Fin d → ℂ) → ℂ} (hP : P ∈ TrueHermitePolys κ) :
10  ∃ M : ℝ, 0 < M ∧
11    ∀ Q ∈ TrueHermiteFunctions κ,
12      ∃ θ : ℂ, ‖θ‖ = 1 ∧
13        ∫ z, ‖P z - θ * Q ‖z ^ 2 ∂ γ
14        ≤ M ^ 2 * ∫ z, ‖(P ‖z - ‖Q ‖z) ^ 2 ∂ γ
```

- Enunciado por **integrais explícitas** (sem teoria abstrata de Hilbert), com a constante M quantificada.
- θ no lugar de λ — reservado para funções anônimas em Lean.

Onde o LLM ajudou (paper B)

Onde ajudou

- **Destravou a direção:** de uma redução *falsa* a uma estimativa de círculo (*Anedota*), vimos o que salvar.
- **Refinou a estimativa:** nosso esboço perdia um log; o GPT (Codex) refinou — saiu a prova completa.

Onde o LLM ajudou (paper B)

Onde ajudou

- **Destravou a direção:** de uma redução *falsa* a uma estimativa de círculo (*Anedota*), vimos o que salvar.
- **Refinou a estimativa:** nosso esboço perdia um log; o GPT (Codex) refinou — saiu a prova completa.

Onde *não* ajudou

- A redução-chave *não* veio do LLM (variante de resultado clássico); no Lean, traduções erradas do blueprint e estrutura de Hilbert desnecessária.

Onde o LLM ajudou (paper B)

Onde ajudou

- **Destravou a direção:** de uma redução *falsa* a uma estimativa de círculo (*Anedota*), vimos o que salvar.
- **Refinou a estimativa:** nosso esboço perdia um log; o GPT (Codex) refinou — saiu a prova completa.

Onde *não* ajudou

- A redução-chave *não* veio do LLM (variante de resultado clássico); no Lean, traduções erradas do blueprint e estrutura de Hilbert desnecessária.

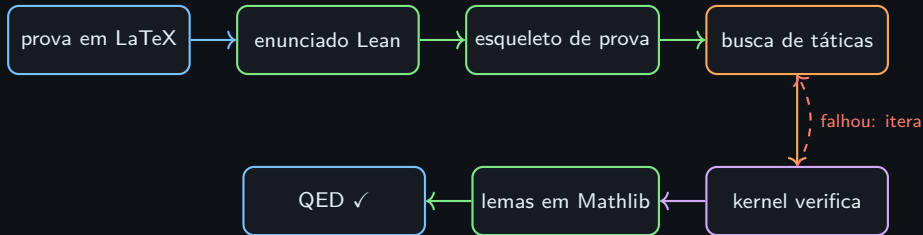
Anedota — a redução falsa que funcionou

Numa conversa de várias trocas com o GPT, surgiu uma **redução incorreta** do problema a uma estimativa de círculo. Insistindo em entendê-la, percebemos que uma *modificação* do argumento se salvava — e levava à prova.

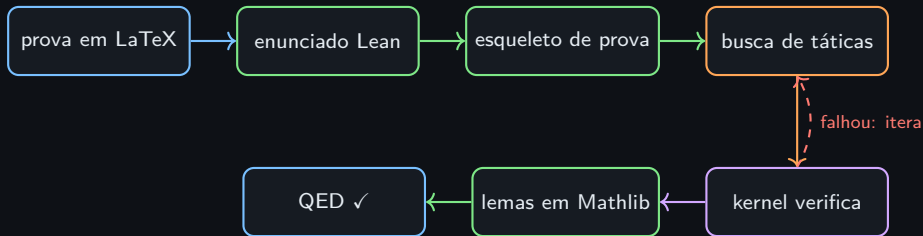
6

Verificação autônoma em Lean

O pipeline de autoformalização



O pipeline de autoformalização



- **Agente** = LLM + ambiente Lean + memória de tentativas.
- Cada passo é **verificado pelo kernel** — nenhuma alucinação sobrevive.

Como formalizamos: o passo-a-passo

1. **Esboço / fatiamento.** Quebrar o paper em pedaços *auto-contidos* (uma ideia por arquivo). Extrair *toda* referência externa, separando “matemática comum” de “resultados citados”. Isolar cada lema com seu contexto mínimo e revisá-lo para correção.

Como formalizamos: o passo-a-passo

1. **Esboço / fatiamento.** Quebrar o paper em pedaços *auto-contidos* (uma ideia por arquivo). Extrair *toda* referência externa, separando “matemática comum” de “resultados citados”. Isolar cada lema com seu contexto mínimo e revisá-lo para correção.
2. **Blueprint.** Transformar a prova numa árvore de arquivos .md. Reescrever os enunciados em forma *finitária e quantitativa*. *Toda* hipótese explícita. Pré-chechar que cada resultado citado existe no Mathlib.

Como formalizamos: o passo-a-passo

1. **Esboço / fatiamento.** Quebrar o paper em pedaços *auto-contidos* (uma ideia por arquivo). Extrair *toda* referência externa, separando “matemática comum” de “resultados citados”. Isolar cada lema com seu contexto mínimo e revisá-lo para correção.
2. **Blueprint.** Transformar a prova numa árvore de arquivos .md. Reescrever os enunciados em forma *finitária e quantitativa*. *Toda* hipótese explícita. Pré-checkar que cada resultado citado existe no Mathlib.
3. **Enxame em Lean.** Um agente por arquivo, cada um editando só sua cópia e importando as outras versões. Dependências entram como *axiomas/sorry* (proposital). Ninguém muda assinatura de teorema sem sinalizar.

Como formalizamos: o passo-a-passo

1. **Esboço / fatiamento.** Quebrar o paper em pedaços *auto-contidos* (uma ideia por arquivo). Extrair *toda* referência externa, separando “matemática comum” de “resultados citados”. Isolar cada lema com seu contexto mínimo e revisá-lo para correção.
2. **Blueprint.** Transformar a prova numa árvore de arquivos .md. Reescrever os enunciados em forma *finitária e quantitativa*. *Toda* hipótese explícita. Pré-checkar que cada resultado citado existe no Mathlib.
3. **Enxame em Lean.** Um agente por arquivo, cada um editando só sua cópia e importando as outras versões. Dependências entram como *axiomas/sorry* (proposital). Ninguém muda assinatura de teorema sem sinalizar.
4. **Kernel + faxina.** Substituir os originais pelas cópias quando *sorry-free*; deixar lemas “Mathlib-ready”, anotando os reaproveitáveis com `-- to_mathlib::`.

Como formalizamos: o passo-a-passo

1. **Esboço / fatiamento.** Quebrar o paper em pedaços *auto-contidos* (uma ideia por arquivo). Extrair *toda* referência externa, separando “matemática comum” de “resultados citados”. Isolar cada lema com seu contexto mínimo e revisá-lo para correção.
2. **Blueprint.** Transformar a prova numa árvore de arquivos .md. Reescrever os enunciados em forma *finitária e quantitativa*. *Toda* hipótese explícita. Pré-checkar que cada resultado citado existe no Mathlib.
3. **Enxame em Lean.** Um agente por arquivo, cada um editando só sua cópia e importando as outras versões. Dependências entram como *axiomas/sorry* (proposital). Ninguém muda assinatura de teorema sem sinalizar.
4. **Kernel + faxina.** Substituir os originais pelas cópias quando *sorry-free*; deixar lemas “Mathlib-ready”, anotando os reaproveitáveis com `-- to_mathlib::`.
5. **Loop.** Devolver os enunciados verificados $\rightarrow$ contribuir ao Mathlib $\rightarrow$ a próxima prova fica mais fácil.

A realidade crua (sem filtro)

- **Tempo:** cada resultado “ingênuo” leva tipicamente ~ 2 dias para formalizar de ponta a ponta.

A realidade crua (sem filtro)

- **Tempo:** cada resultado “ingênuo” leva tipicamente ~ 2 dias para formalizar de ponta a ponta.
- **Custo:** consome boa parte da cota mensal de tokens de uma conta **Claude Max / GPT Max** — não é barato.

A realidade crua (sem filtro)

- **Tempo:** cada resultado “ingênuo” leva tipicamente ~ 2 dias para formalizar de ponta a ponta.
- **Custo:** consome boa parte da cota mensal de tokens de uma conta **Claude Max / GPT Max** — não é barato.
- **Axiomas implícitos:** o primeiro passe quase sempre “fecha” — mas escondendo sorry/axiomas nas dependências. **Auditar sempre** (`#print axioms`).

A realidade crua (sem filtro)

- **Tempo:** cada resultado “ingênuo” leva tipicamente ~ 2 dias para formalizar de ponta a ponta.
- **Custo:** consome boa parte da cota mensal de tokens de uma conta **Claude Max / GPT Max** — não é barato.
- **Axiomas implícitos:** o primeiro passe quase sempre “fecha” — mas escondendo sorry/axiomas nas dependências. **Auditar sempre** (`#print axioms`).
- **Refinar é a regra:** um enunciado ingênuo vira um enunciado *difícil* em Lean. Trocar `tsum`/integrais por somas finitas, caçar `MAX_HEARTBEATS`, requantificar constantes.

A realidade crua (sem filtro)

- **Tempo:** cada resultado “ingênuo” leva tipicamente ~ 2 dias para formalizar de ponta a ponta.
- **Custo:** consome boa parte da cota mensal de tokens de uma conta **Claude Max / GPT Max** — não é barato.
- **Axiomas implícitos:** o primeiro passe quase sempre “fecha” — mas escondendo sorry/axiomas nas dependências. **Auditar sempre** (`#print axioms`).
- **Refinar é a regra:** um enunciado ingênuo vira um enunciado *difícil* em Lean. Trocar tsum/integrais por somas finitas, caçar MAX_HEARTBEATS, requantificar constantes.
- **O ciclo real:** refinar $\rightarrow$ rebuildar $\rightarrow$ reauditar. O “co-autor” não é autônomo — é um operário rápido que precisa de **andaimes humanos**.

A realidade crua (sem filtro)

- **Tempo:** cada resultado “ingênuo” leva tipicamente **~2 dias** para formalizar de ponta a ponta.
- **Custo:** consome boa parte da cota mensal de tokens de uma conta **Claude Max / GPT Max** — não é barato.
- **Axiomas implícitos:** o primeiro passe quase sempre “fecha” — mas escondendo sorry/axiomas nas dependências. **Auditar sempre** (`#print axioms`).
- **Refinar é a regra:** um enunciado ingênuo vira um enunciado *difícil* em Lean. Trocar `tsum`/integrais por somas finitas, caçar `MAX_HEARTBEATS`, requantificar constantes.
- **O ciclo real:** refinar → rebuildar → reauditar. O “co-autor” não é autônomo — é um operário rápido que precisa de **andaimes humanos**.

Lento e caro *hoje*. Mas a certeza é **absoluta**, e a biblioteca composta acelera a próxima prova.

O que o agente fez bem (e mal)

Bem

- Encontrou lemas no Mathlib que nós *não conhecíamos*.
- Resolveu pequenas imprecisões (tipo, namespace, coerção).
- Iterou rapidamente quando `simp/linarith` quase fechavam.
- Aprendeu a estruturar blocos longos.

O que o agente fez bem (e mal)

Bem

- Encontrou lemas no Mathlib que nós *não conhecíamos*.
- Resolveu pequenas imprecisões (tipo, namespace, coerção).
- Iterou rapidamente quando simp/linarith quase fechavam.
- Aprendeu a estruturar blocos longos.

Mal

- Inventou nomes de lemas que não existiam.
- Misturou Lean 3 e Lean 4 (begin...end vs by).
- Em provas longas, perdeu o “estado” do goal.
- Falhou em raciocínios verdadeiramente *globais* — precisa de **andaimes humanos**.

O que o agente fez bem (e mal)

Bem

- Encontrou lemas no Mathlib que nós *não conhecíamos*.
- Resolveu pequenas imprecisões (tipo, namespace, coerção).
- Iterou rapidamente quando simp/linarith quase fechavam.
- Aprendeu a estruturar blocos longos.

Mal

- Inventou nomes de lemas que não existiam.
- Misturou Lean 3 e Lean 4 (begin...end vs by).
- Em provas longas, perdeu o “estado” do goal.
- Falhou em raciocínios verdadeiramente *globais* — precisa de **andaimes humanos**.

Resultado líquido: **muito útil**, especialmente quando o matemático sabe *o que pedir e como fazer*.

7

Perspectiva

O que muda no trabalho do matemático?

- **Curto/Médio prazo:** co-piloto onipresente; menos trabalho braçal; formalização *em paralelo* com a escrita; busca de lemas e contas verificadas.
- **Longo prazo:** ?

O que muda no trabalho do matemático?

- **Curto/Médio prazo:** co-piloto onipresente; menos trabalho braçal; formalização *em paralelo* com a escrita; busca de lemas e contas verificadas.
- **Longo prazo:** ? Por enquanto, é só isso que podemos afirmar - não sabemos para onde o longo prazo vai, e prometer certezas = chute.

O que muda no trabalho do matemático?

- **Curto/Médio prazo:** co-piloto onipresente; menos trabalho braçal; formalização *em paralelo* com a escrita; busca de lemas e contas verificadas.
- **Longo prazo:** ? Por enquanto, é só isso que podemos afirmar - não sabemos para onde o longo prazo vai, e prometer certezas = chute.

Uma coisa não muda: quem **atribui significado, conecta ideias distantes, enxerga beleza e decide o que vale a pena perguntar** somos nós. Os LLMs são, e seguirão sendo, *ferramentas excelentes*; o trabalho matemático permanece, no fundo, **inerentemente humano**.

Direções futuras concretas

Lean & autoformalização

- Contribuir os lemas de recuperação de fase ao Mathlib.
- Tornar reprodutível o pipeline *artigo* $\rightarrow$ *Lean*.
- *Benchmarks* de autoformalização em análise harmônica.

Direções futuras concretas

Lean & autoformalização

- Contribuir os lemas de recuperação de fase ao Mathlib.
- Tornar reproduzível o pipeline *artigo* $\rightarrow$ *Lean*.
- *Benchmarks* de autoformalização em análise harmônica.

LLMs & aplicações

- Melhorar a tradução *blueprint* $\rightarrow$ *Lean* (o gargalo de hoje).
- Busca de provas assistida por agentes, com auditoria de axiomas.
- Levar os resultados de volta à recuperação de fase *experimental* (e.g. amostragem em espiral).

Direções futuras concretas

Lean & autoformalização

- Contribuir os lemas de recuperação de fase ao Mathlib.
- Tornar reproduzível o pipeline *artigo* $\rightarrow$ *Lean*.
- *Benchmarks* de autoformalização em análise harmônica.

LLMs & aplicações

- Melhorar a tradução *blueprint* $\rightarrow$ *Lean* (o gargalo de hoje).
- Busca de provas assistida por agentes, com auditoria de axiomas.
- Levar os resultados de volta à recuperação de fase *experimental* (e.g. amostragem em espiral).

"I think this is going to be very transformative for mathematics. ... Mathematicians and AI will work together."

— Terence Tao

Obrigado

Obrigado!

Trabalho em colaboração com
P. Abdalla, S. Bertolini, J. de Dios, B. Pineau, M. Taylor.

“The book of nature is written in the language of mathematics.” — Galileo

“... and increasingly, mathematics is being written in the language of Lean.”

questions? # perguntas?